OPERATING SYSTEMS

Course Objectives:

- Learn basics of operating Systems.
- Understand the process management and synchronization that take place in the operating system.
- Learn the principles of memory, I/O and file management in a secured environment

COURSE OUTCOMES:

At the end of this course the student will be able to

- Understand the basic functionalities and structure of the Operating System
- Gets familiar the concepts and implementations of Processes, Process Scheduling. Describe, contrast and compare various types of Operating systems like Windows and Linux.
- Comprehend the concepts of Synchronization and Deadlocks in the Operating System
- Understand the concepts of Memory Management(Physical and Virtual memory)
- Understands the concepts of File System with regard to directory and disk management algorithms.
- Understand the concepts of I/O systems, protection and security in a case study given

Syllabus

- **UNIT I :** Introduction to Operating System
- **UNIT II:** Process Management
- **UNIT III:** Process-Synchronization
- **UNIT IV**: Deadlocks & Memory Management
- **UNIT V:** File System
- **UNIT VI:** I/O Systems

UNIT–I: Introduction

1. Introduction to Operating System Concepts

- **Multitasking**
- **Multiprogramming**
- **Multiuser**
- **Multithreading**
- **…**

2. Types of Operating Systems

- **Batch Operating System**
- **Time–sharing Systems**
- **Distributed OS**
- **Network OS**
- **Real–Time OS**

3. Various Operating System Services

4. Architecture

5. System Calls and Programs

Exam Questions

Prerequisites

|<<

- **Basic Data Structures**
- **Computer Organisation**
- **High–Level Language (C, …)**

Operating System (OS)

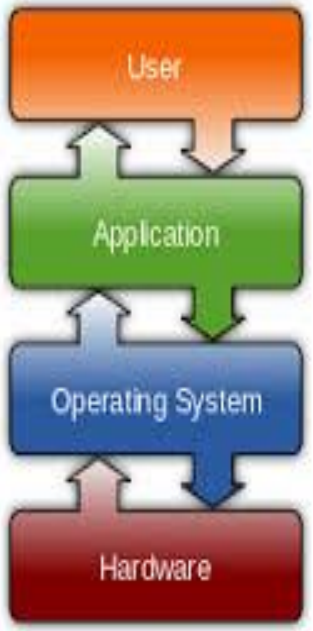
Defn1: **OS** is software that manages computer hardware and software resources and provides common services for computer programs. (source: Wiki)

Defn2: **Controls and coordinates use of Hardware among Various Applications and Users**

Defn3:**Collection of System Software (Programs), which makes it User friendly and Most efficient**

Kernel

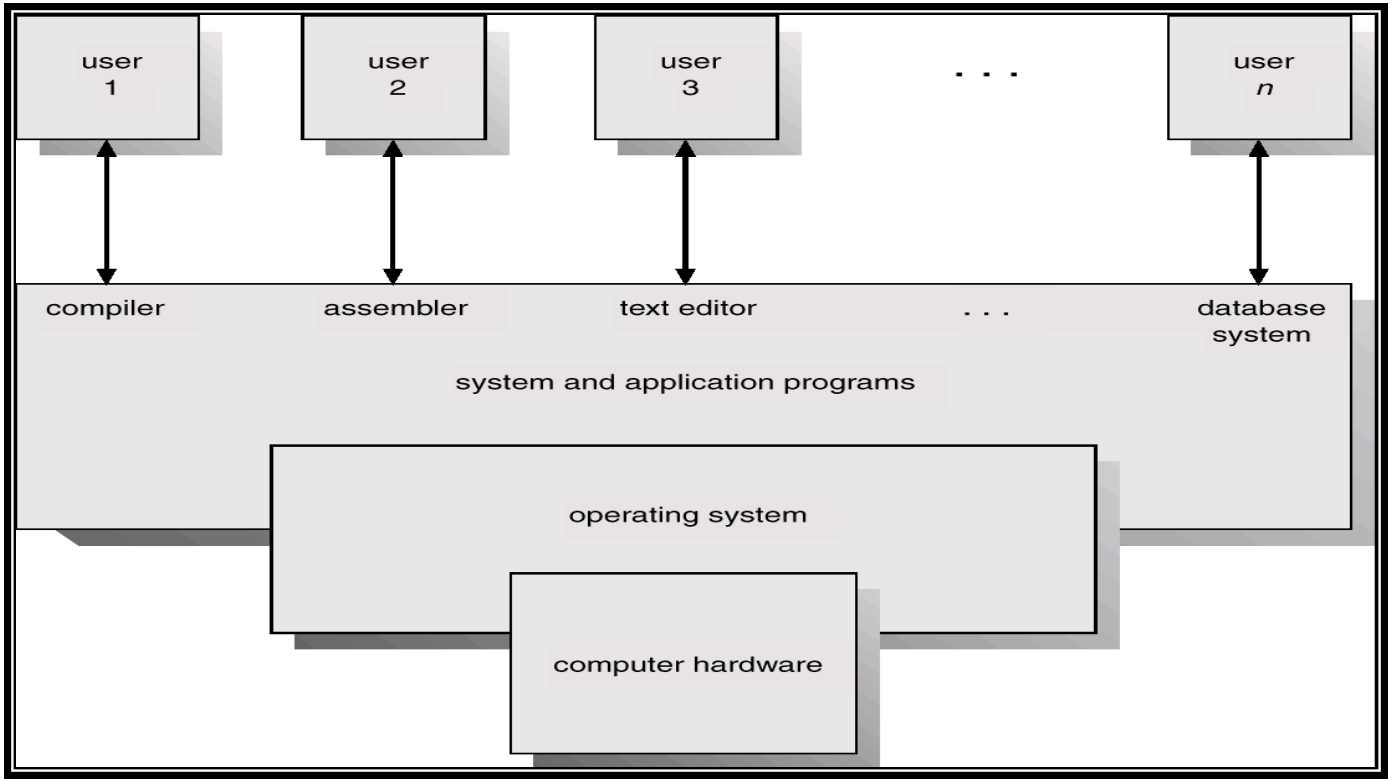
Program running at all times on the Computer



Kernel

- Central module and Part of the OS that loads first, and it remains in main memory.
- So, it has to be as small as possible while still providing all the essential services required by other parts of the operating system and applications.
- Its loaded into a protected area of memory to prevent it from being overwritten by programs or other parts of the operating system.
- Responsible for memory management, process and task management, and disk management.
  - The kernel connects the system hardware to the application software.
- Every OS has a kernel.

Basic Structure/Components of a Computer System



- **Hardware** →
  1. **Central Processing Unit (CPU)**
  2. **Memory**
  3. **Input–Output (I/O) / Peripheral Devices**

  • **Provides basic Computing Resources.**
- **Operating System**
- **Application Programs**
- **Users**

User View

The user viewpoint focuses on how the user interacts with the operating system through the usage of various application programs

Varies according to the interface used

Goal

To maximize the work, the user is performing

- **Ease of use**
- **Performance**
- **Resource utilization**

Mainframe / Minicomputer

- **Share Resources**
- **Exchange Information**
- **Designed to maximize Resource utilization**
  - ✓**All Available CPU Time, Memory and I/O used efficiently**

User View (contd.)

- **Workstations (Perform Effective Resource Sharing)**
  - **Connected to Networks of other workstations and Servers**
  - **Dedicated Resources at disposal**
  - **Also Share Resources such as Networking and Servers (File, Compute, Print Servers)**

- **Handheld Computers (Ease of use)**
  - **Standalone units for Individual users**
  - **Connected to Networks (Directly by wire / through wireless Modems and Networking)**
  - **Perform relatively few remote operations (Power, Speed, Interface limitations)**
  - **Designed mostly for individual usability**

- **Embedded Computers in Home Devices and Automobiles (Performance)**
  - **Have Numeric Keypads**
  - **May turn indicator lights on or off to show status**
  - **Designed primarily to run without user intervention**

System View

The system viewpoint focuses on how the hardware interacts with the operating system to complete various tasks.

Most intimately involved with Hardware

- **Resource Allocator**
  - Acts as Manager of Resources.
  - Decides to allocate Resources to Specific Programs and Users so that OS operates the Computer System Efficiently and Fairly
  - Important as many Users access the same Mainframe / Minicomputer
- **Control Program**
  - Manages Execution of User Programs to prevent Errors and Improper use of Computer
  - Concerned with Operation and Control of I/O Devices

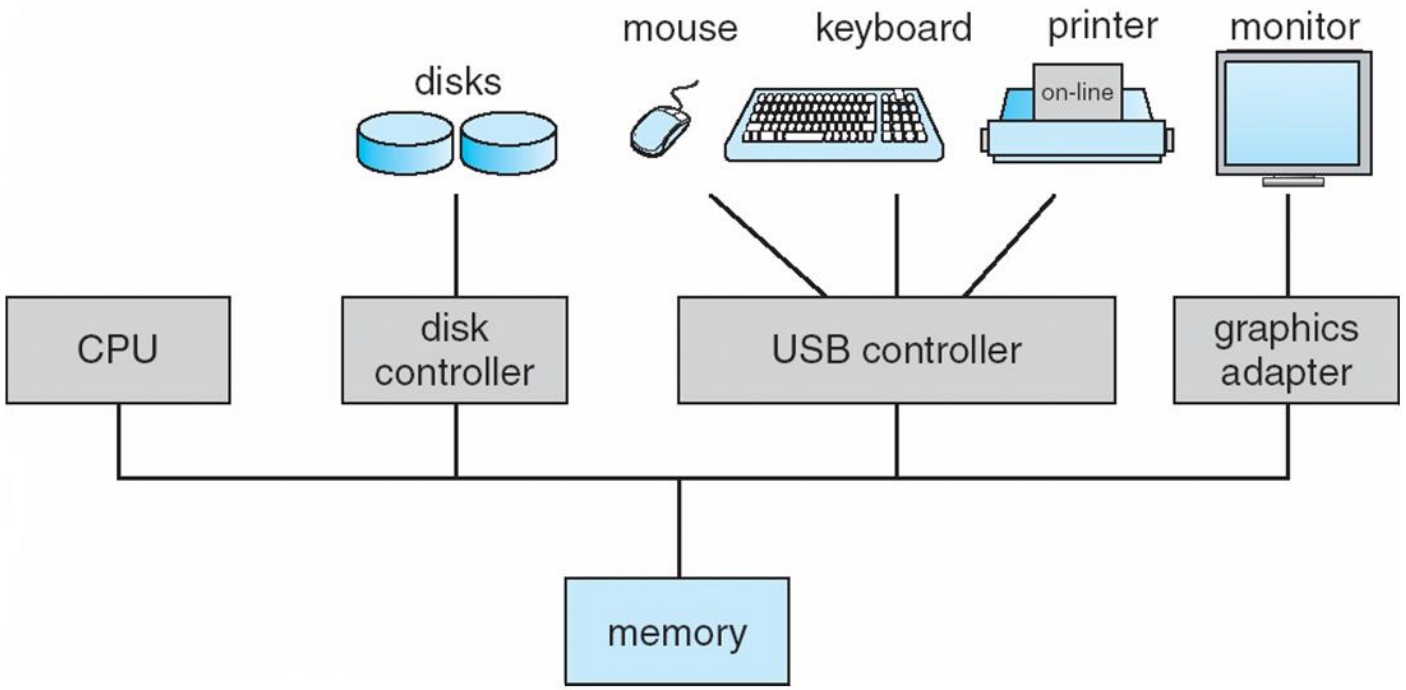
Objectives and Functions of Operating System

Essential part of a Computer System

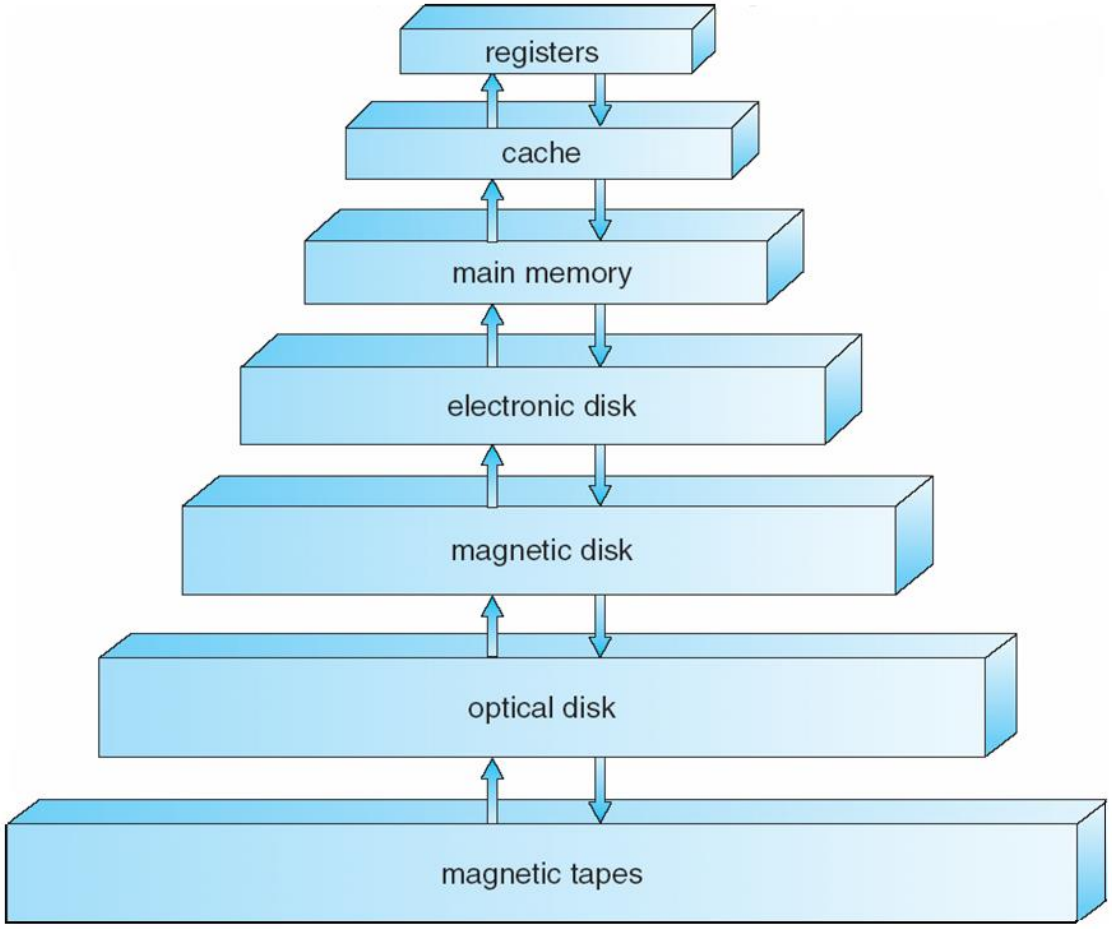
- Purpose: To provide an environment in which a User can Execute Programs in a *Convenient* and *Efficient manner*
- Program that acts as an intermediary between the User of a Computer and the Computer Hardware
  - Must Ensure Correct Operation of Computer System
  - Provides Certain Services to Programs and users of those programs in order to make their Tasks Easier
  - Controls and Coordinates the use of Hardware among Various Application Programs for the Various users
  - Primary Goals-
    User friendly
    Efficient Operation of the Computer System

Computer System Organization

- Computer-system operation
  - One or more CPUs, device controllers connect through common bus providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



Storage-Device Hierarchy



- The higher levels are expensive, but are fast.
- As we move down the hierarchy, the cost per bit decreases, where as Access time generally increases.

Computer-System Architecture

1. Single-Processor Systems
2. Multi-Processor Systems
3. Clustered Systems

Computer-System Architecture

- Single Processor Systems
  - One CPU
  - May have other special-purpose processors (such as disk controllers)
    - Run a limited instruction set
    - Do not run user processes
    - Sometimes managed by OS
    - For example a disk controller processor receives a sequence of requests from the main CPU and implements its own disk queue and scheduling algorithm to relieve the main CPU of the overhead of disk scheduling.

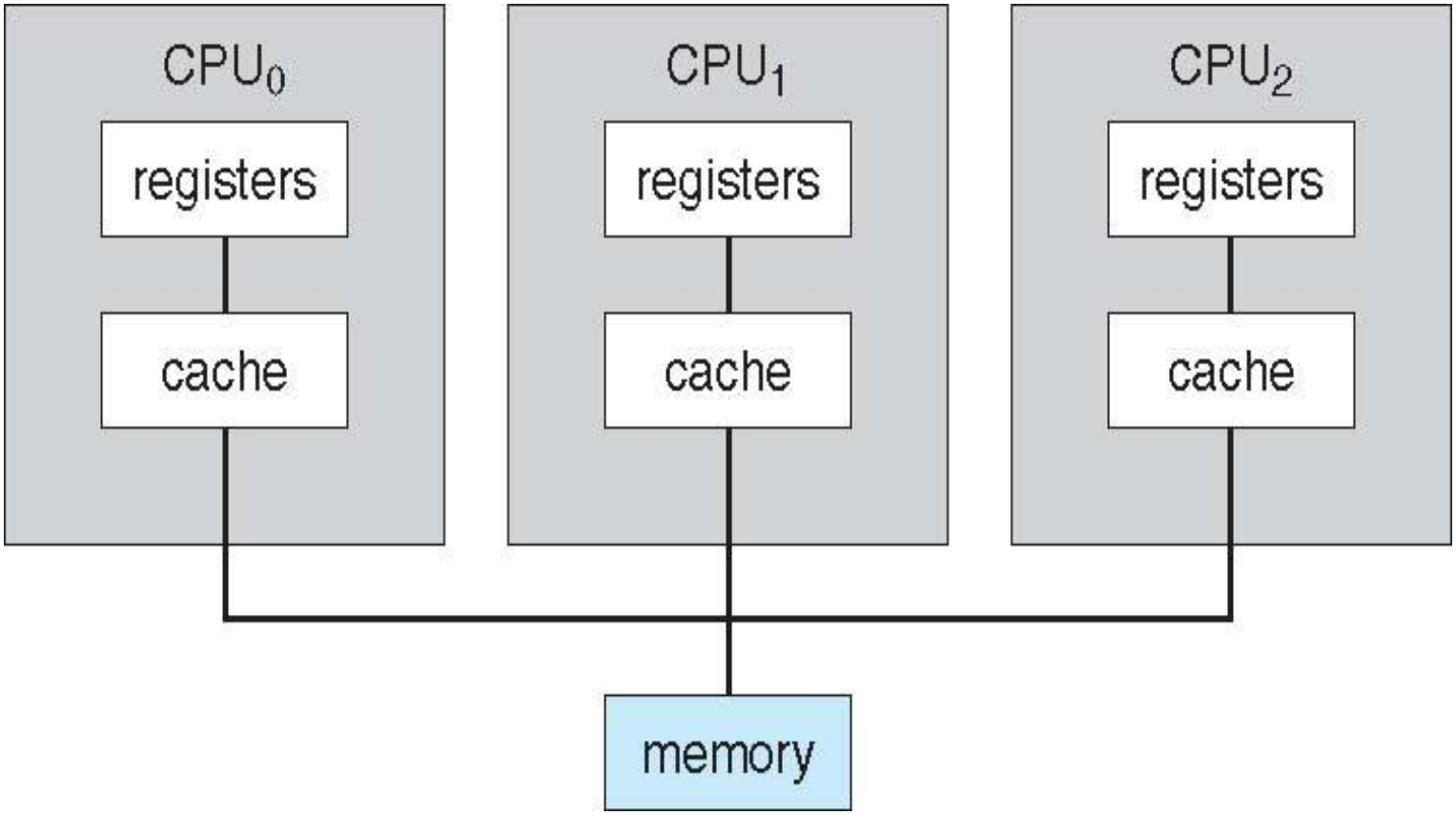
Multiprocessor systems

- Two or more processors in close communication, sharing the computer bus and sometimes the clock, memory, and peripherals.
- Advantages:
  - Increased throughput
    - The speed-up ratio with N processors is not N however, it is less than N.
  - Economy of scale
    - Sharing of peripherals, mass storage, power supplies.
    - Can cost less than multiple single-processor systems
  - Increased reliability
    - Graceful degredation(The ability to continue providing service proportional to the level of surviving hardware)
    - Fault tolerance
      - Failure detection, diagnose and correction

Multiprocessor Systems

- Two types:
  - **Asymmetric multiprocessing** in which each processor is assigned a specific task. A master processor controls the system, scheduling and allocating work to slave processors.
  - **Symmetric multiprocessing (SMP)** in which each processor performs all tasks within the operating system. All processors are peers.

Symmetric Multiprocessing Architecture

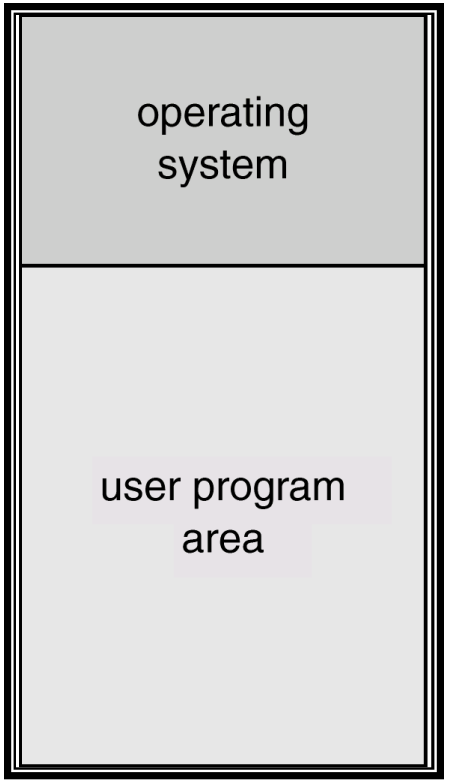


Clustered Systems

- Gather together multiple CPUs to accomplish computation
- Composed of two or more individual systems coupled together.
- **High availability** service.
  - Each node can monitor one or more of the others over the LAN.
  - If the monitored machine fails, the monitoring machine can take ownership of its storage and restart the applications that were running on the failed machine.
- Structure
  - **Asymmetric clustering:** One machine is in hot-standby mode while the other is running applications. Hot-standby machine only monitors the active server.
  - **Symmetric clustering:**Two or more hosts are running applications and are monitoring each other
- **Parallel Clusters** allow multiple hosts to access the same data on the shared storage. May need a distributed lock manager (DLM).

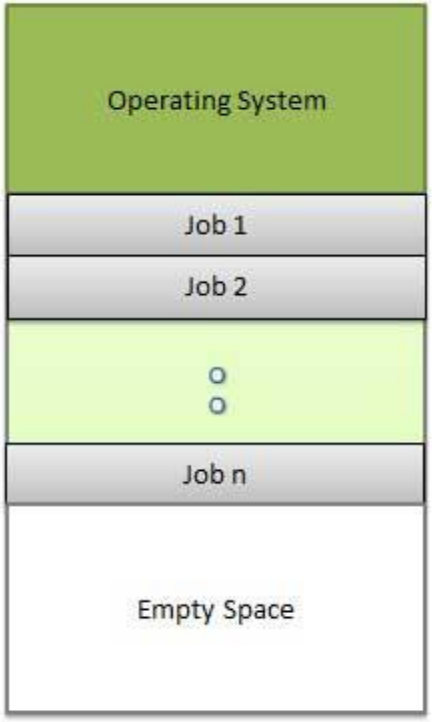
Uniprogramming

- **Single User System**
- **Single Program in execution.**



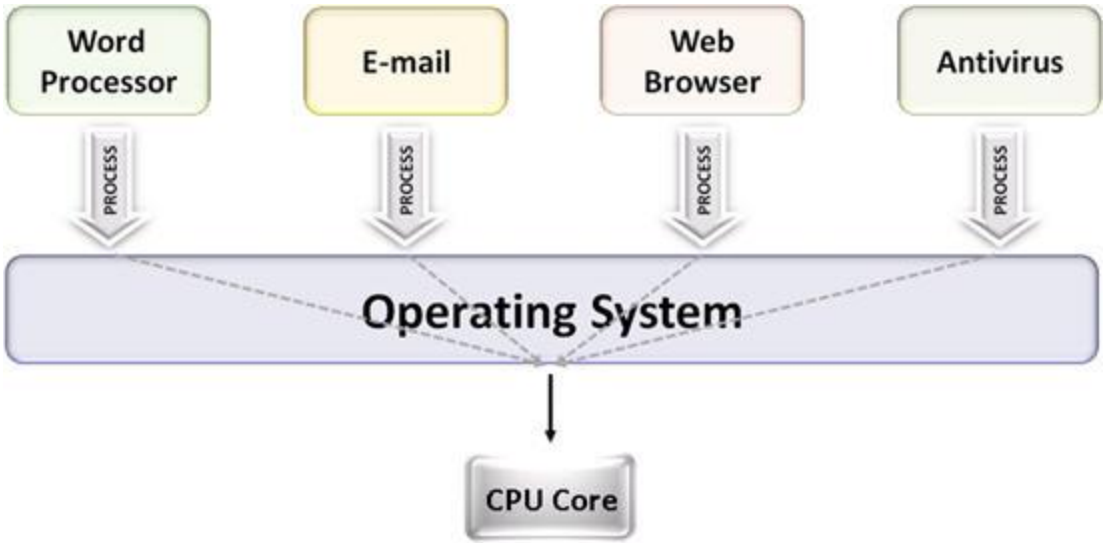
Multiprogramming Systems

Multiprogramming is also the ability of an operating system to execute more than one program on a single processor machine. More than one task/program/job/process can reside into the main memory at one point of time. A computer running excel and firefox browser simultaneously is an example of multiprogramming.



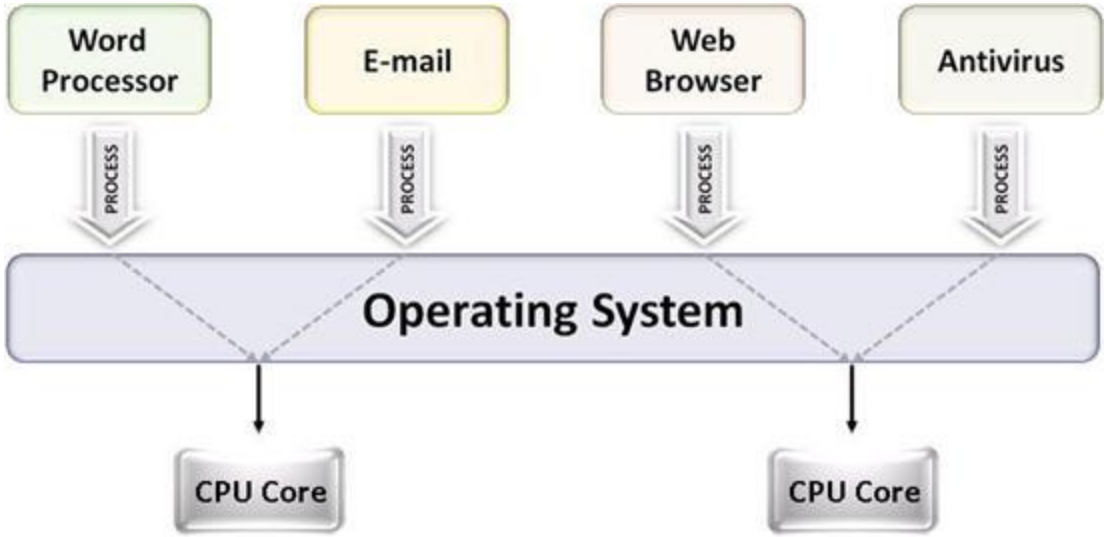
Multitasking

Multitasking is the ability of an operating system to execute more than one task simultaneously on a single processor machine. Though we say so but in reality no two tasks on a single processor machine can be executed at the same time. Actually CPU switches from one task to the next task so quickly that appears as if all the tasks are executing at the same time. More than one task/program/job/process can reside into the same CPU at one point of time.



Multiprocessing

Multiprocessing is the ability of an operating system to execute more than one process simultaneously on a multi processor machine. In this, a computer uses more than one CPU at a time.



Multiuser

Allows for Multiple users to use the Same Computer in timely manner.

Multithreading

Multithreading is the ability of an operating system to execute the different parts of a program called threads at the same time. Threads are the light wait processes which are independent part of a process or program. In multithreading system, more than one threads are executed parallely on a single CPU.

Time Sharing

|<<

Time sharing is a logical extension of multiprogramming.
In this, the CPU executes multiple jobs by switching among them, but the switches occur so frequently that the users can interact with each program while it is running.

Response time should be < 1 second

Each user has at least one program executing in memory ⇨**process**

Job scheduling: Which jobs to bring to memory from job pool on disk.

If several jobs ready to run at the same time ⇨ **CPU scheduling**

If processes don't fit in memory, **swapping** moves them in and out to run

Types of Operating Systems

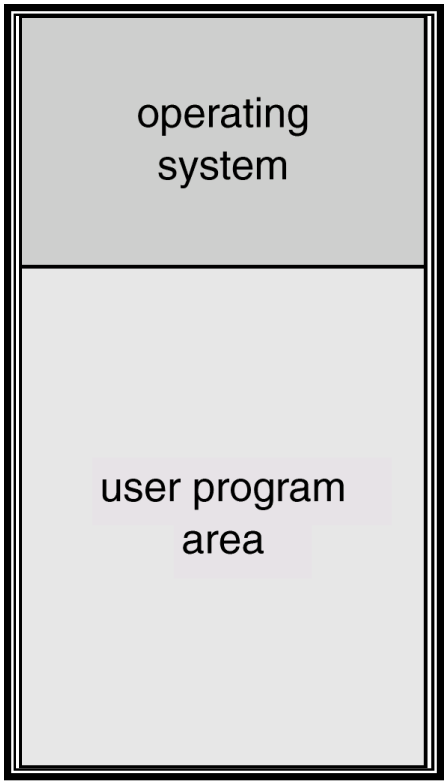
|<<

- **Batch Processing Systems**
- **Interactive Systems**
- **Time Sharing**
- **Time Slicing**
- **Distributed Systems**
- **Network Systems**
- **Real–Time Systems**
- **Evolution of OS**

Batch Processing Systems

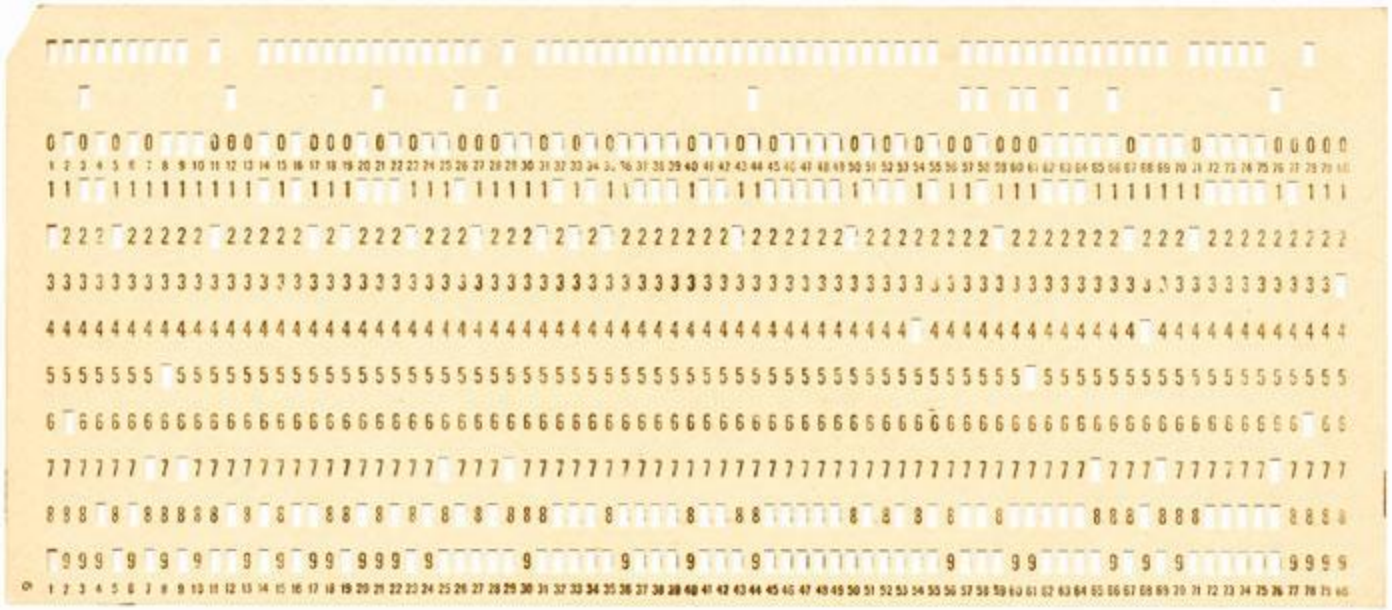
Computer runs one and only one application at a time.

In Batch processing same type of jobs batch *(BATCH- a set of jobs with similar needs)* together and execute at a time.
The OS was simple, its major task was to transfer control from one job to the next.
The job was submitted to the computer operator in form of punch cards. At some later time the output appeared.
The OS was always resident in memory.
Common Input devices were card readers and tape drives.



Common output devices were Line printers ,Tape drives
Card punches

Example of a punch card



ComputerHope.com

Batch Operating System

- The problems with Batch Systems are following.
  - Lack of interaction between the user and job.
  - CPU is often idle, because the speeds of the mechanical I/O devices is slower than CPU.
  - Difficult to provide the desired priority

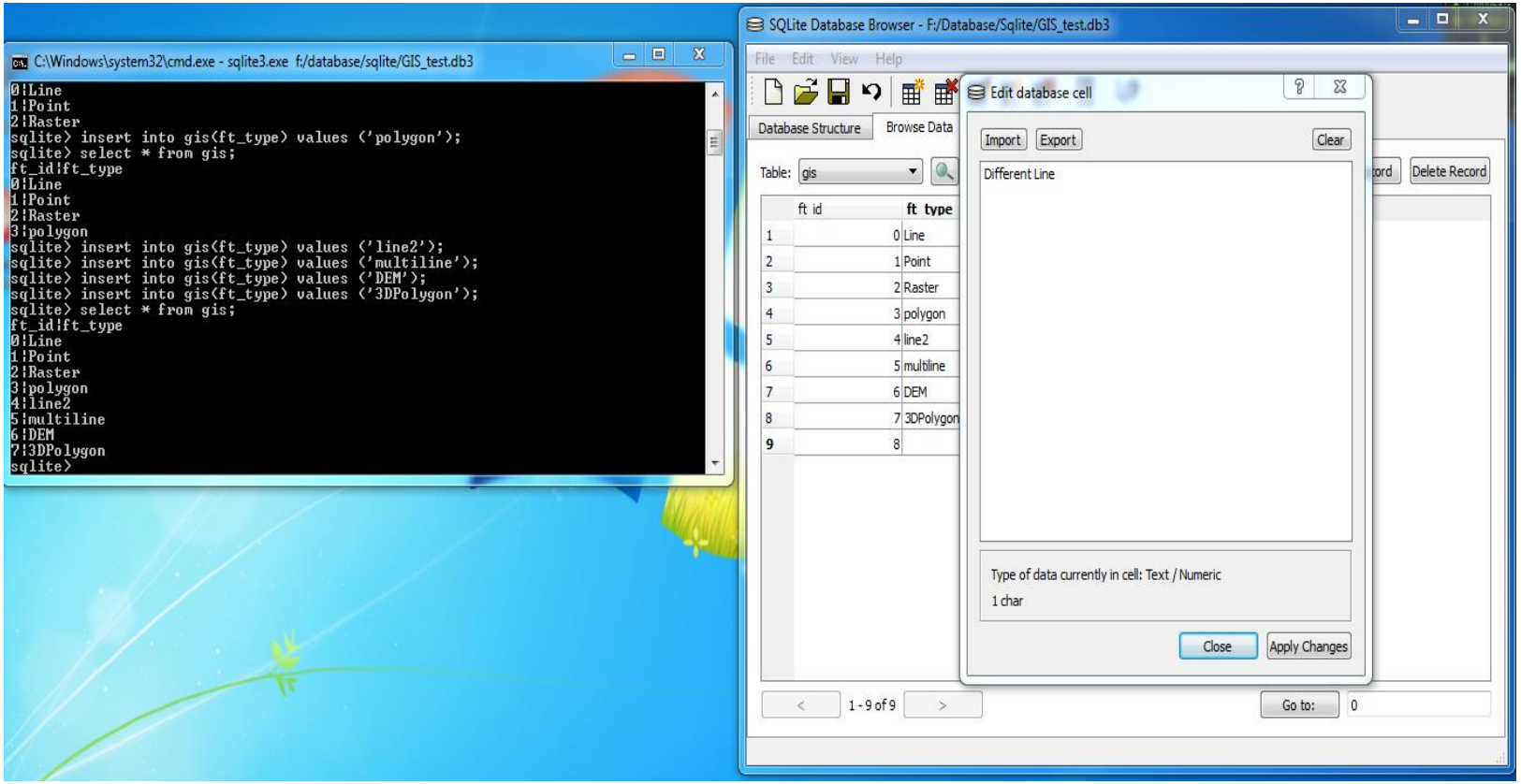
courtesy Argonne National Laboratory

Interactive Processing

|<<

- **The User interacts directly with OS to supply commands and data as the application program executes and the user receives results of processing immediately.**
- **User and Computer System interact.**
- **User Requests.**
- **System Responds.**
- **The process goes on.**

- **There will be an user interface in place to allow all these to happen**
- **It can be CLI or GUI**



Time Sharing

|<<

Variable CPU Time to different processes.

Once CPU Time is allocated to a process, will not be interrupted till it completes its execution, or waiting for an I/O, or cannot continue its execution.

Time Slicing

Equal Amount of CPU time allocated among Various Processes.

Distributed Systems

- Distribute the Computation among Several Physical Processors
- A Distributed Operating System refers to a model in which applications run on multiple interconnected computers, offering enhanced communication and integration capabilities compared to a network operating system.
- In a Distributed Operating System, multiple CPUs are utilized, but for end- users, it appears as a typical centralized operating system. It enables the sharing of various resources such as CPUs, disks, network interfaces, nodes, and computers across different sites, thereby expanding the available data within the entire system.

Loosely Coupled System

•Each Processor has its Own Local Memory.

•Processors Communicate with one another through Various Communications lines, such as High–Speed buses or Telephone lines

•Enables *Parallelism* but speed up is not the goal

Advantages of Distributed Systems

- Resources Sharing
- Computation Speed up–load Sharing
- Reliability
- Communications

Network Operating Systems

- Runs on a server and provides server the capability to manage data, users, groups, security, applications and other networking functions.
- The primary purpose of the network operating system is to allow shared file and printer access among multiple computers in a network
- Egs: Microsoft Windows Server 2003, 2008, UNIX, Linux, Mac OS X

Network Operating Systems: Advantages

- Centralized servers are highly stable
- Security is server managed
- Upgrades to new technologies and hardware can be easily integrated into the system
- Remote access to server is possible

Network Operating Systems: Disadvantages

- High cost
- Regular maintenance and updates are required
- Dependency on central location for most operations

Real–Time Systems

Special purpose OS.

Used when rigid time requirements have been placed on the operation of a processor or the flow of data.

Used as control device in a dedicated application.

Sensors bring data to the computer.

The computer analyses the data and possibly adjust controls to modify the sensor inputs.

Well-defined, Fixed-time constraints.

Processing must be done within the defined constraints.

Ex: Systems controlling scientific experiments
Medical imaging systems
Industrial control systems
Certain display systems
Automobile-engine fuel-injection systems
Home-appliance controllers
Weapon systems

Types of Real–Time Systems

|<<

Hard real-time systems

Guarantee critical tasks be completed on time.

Soft real-time systems

Critical real-time task gets priority over other tasks and retains the priority until it completes.

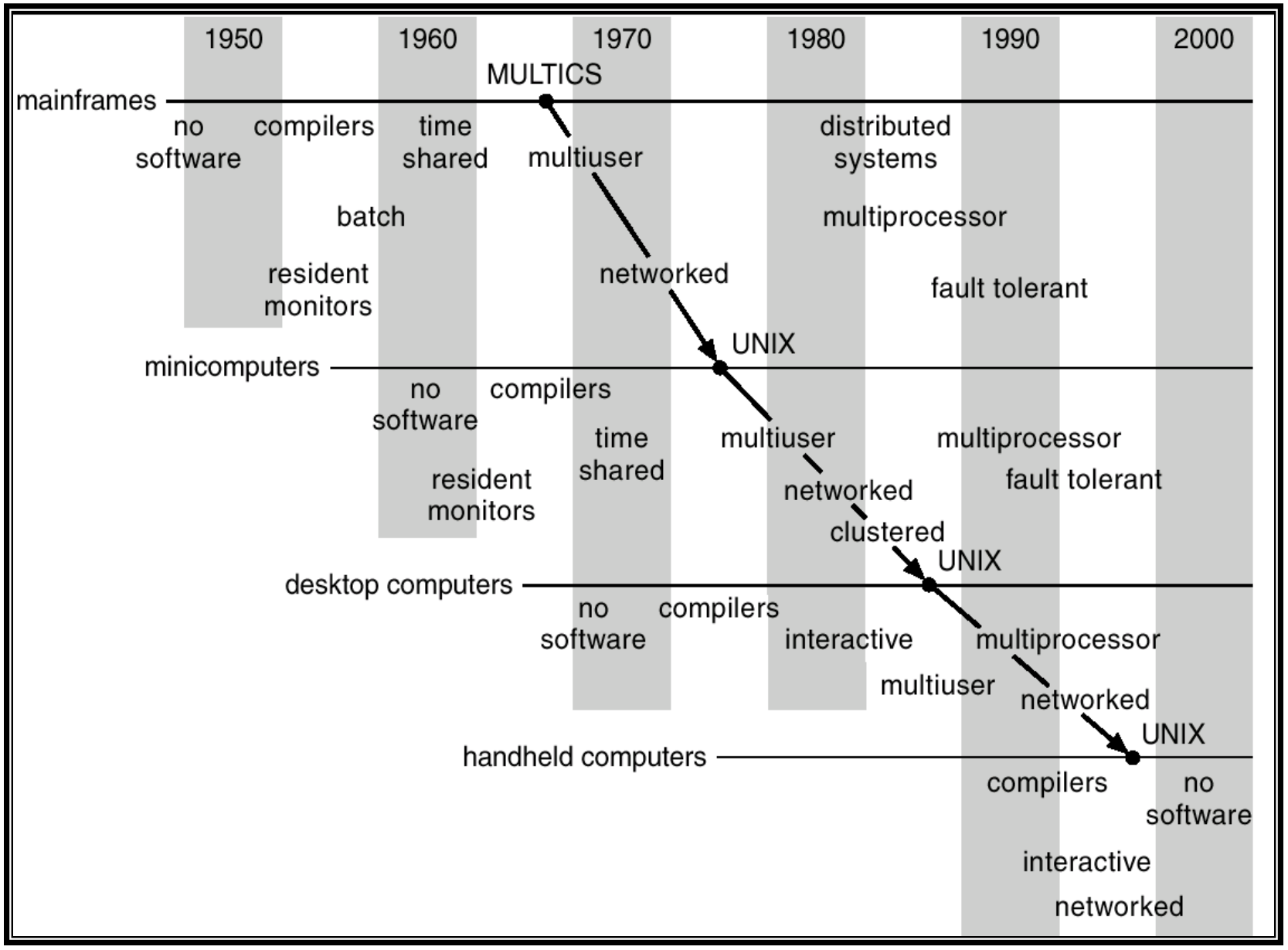
Ex: Multimedia
Advanced scientific projects
Undersea exploration
Planetary rovers

Real Time Operating System

- Mass use in Electronic Devices.
- User Interface is very small

Such as X-ray, city –Scan,etc

Evolution of Operating Systems



Various Operating System Services

|<<

Provides No. of Services.

***Lowest level* – System Calls**

Allow running program to make requests from the OS directly.

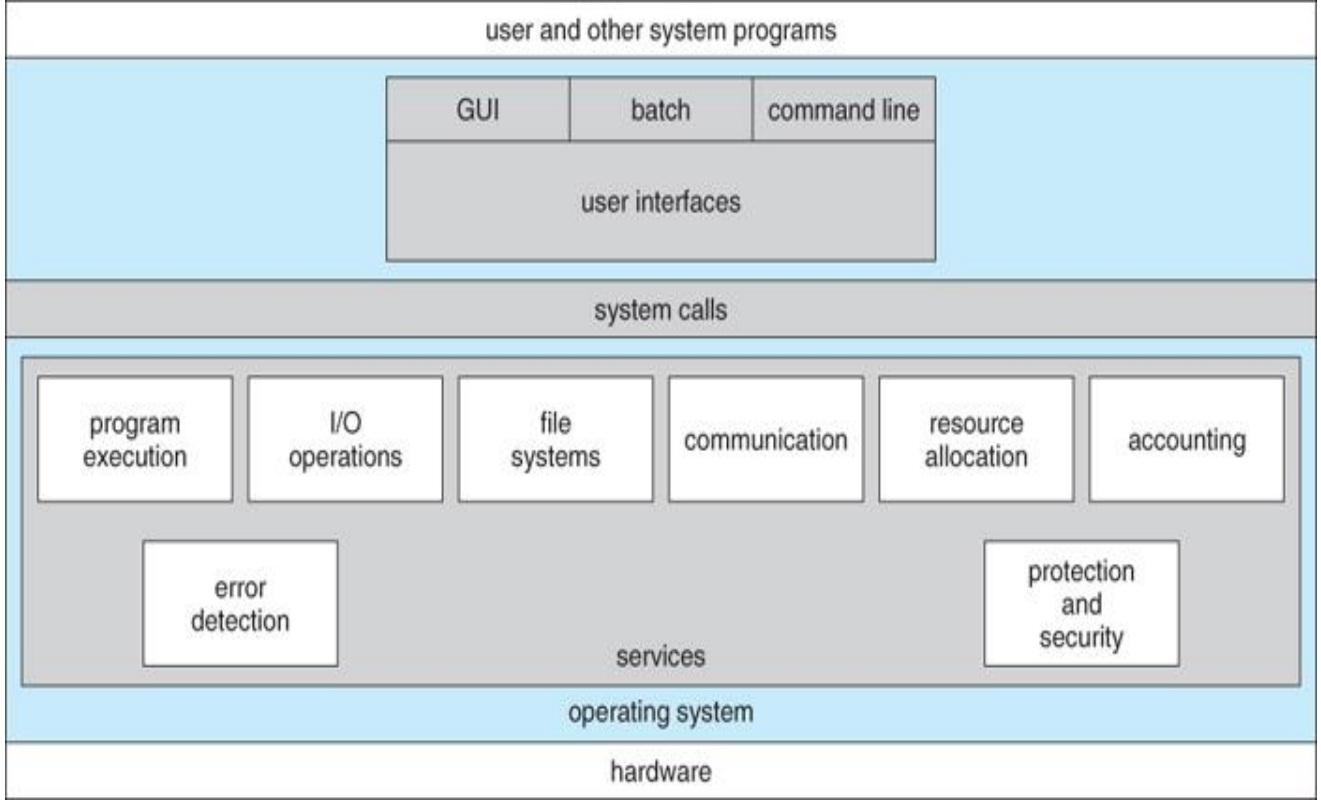
***Higher level* – Command Interpreter or Shell**

User issues request without writing program.

Categories

1. **Program Control**
2. **Status Requests**
3. **I/O Requests**

Operating System Services



|<<

1. User Interface
2. Program Execution
3. I/O Operations
4. File-System Manipulation
5. Communications
6. Error Detection

– **Other Services**

1. Resource Allocation
2. Accounting
3. Protection and Security

User Interface

|<<

- **All Operating Systems have User Interface (UI)**
- **Command-Line Interface (CLI)**
- **Graphics User Interface (GUI)**

Program Execution

- **Load Program into Memory**
- **Run the Program**
- **End Execution, either Normally or Abnormally (indicating Error)**

I/O Operations

|<<

Running Program may require I/O,

which may involve File or I/O Device.

File–System Manipulation

Programs need to

Read and Write Files and Directories,

Create and Delete Files and Directories,

Search Files and Directories,

List File Information,

Permission Management.

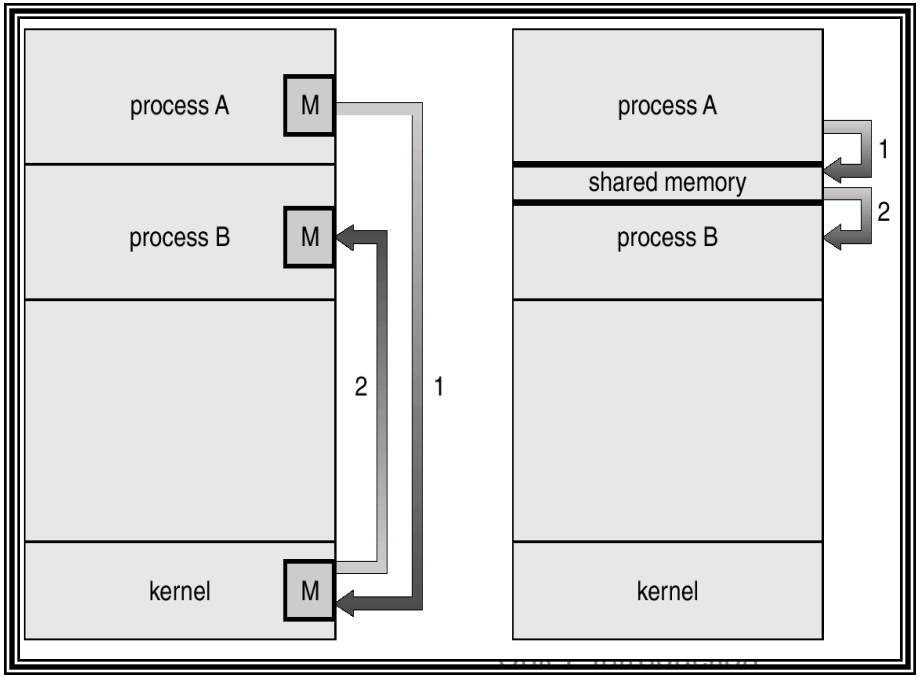
Communications

|<<

Processes may exchange information,
on the Same Computer or between Computers
over a Network

Communications may be via

- **Shared Memory or**
- **thru Message Passing (Packets moved by the Operating System)**



Two ways of passing data between programs.

Error Detection

|<<

- **May occur in**
  - **the CPU and Memory Hardware,**
  - **I / O Devices,**
  - **User Program**

- **For each type of Error,**
  - **OS should take the appropriate action to ensure correct and consistent computing**

- **Debugging facilities**
  - **can greatly enhance the User's and Programmer's abilities to efficiently use the System**

Resource Allocation

When Multiple Users or Multiple Jobs running Concurrently,

Resources must be Allocated to each of them.

Accounting

To Keep Track of

Which Users use How much

What Kinds of Computer Resources

To know Usage statistics.

Protection and Security

- **Protection**
  - **Involves ensuring that all Access to System Resources is Controlled**

- **Security of the System**
  - **from outsiders requires User Authentication,**
  - **extends to defending External I / O Devices from Invalid Access attempts**

System Calls

|<<

- System Calls
  - Types of System Calls
- System Programs

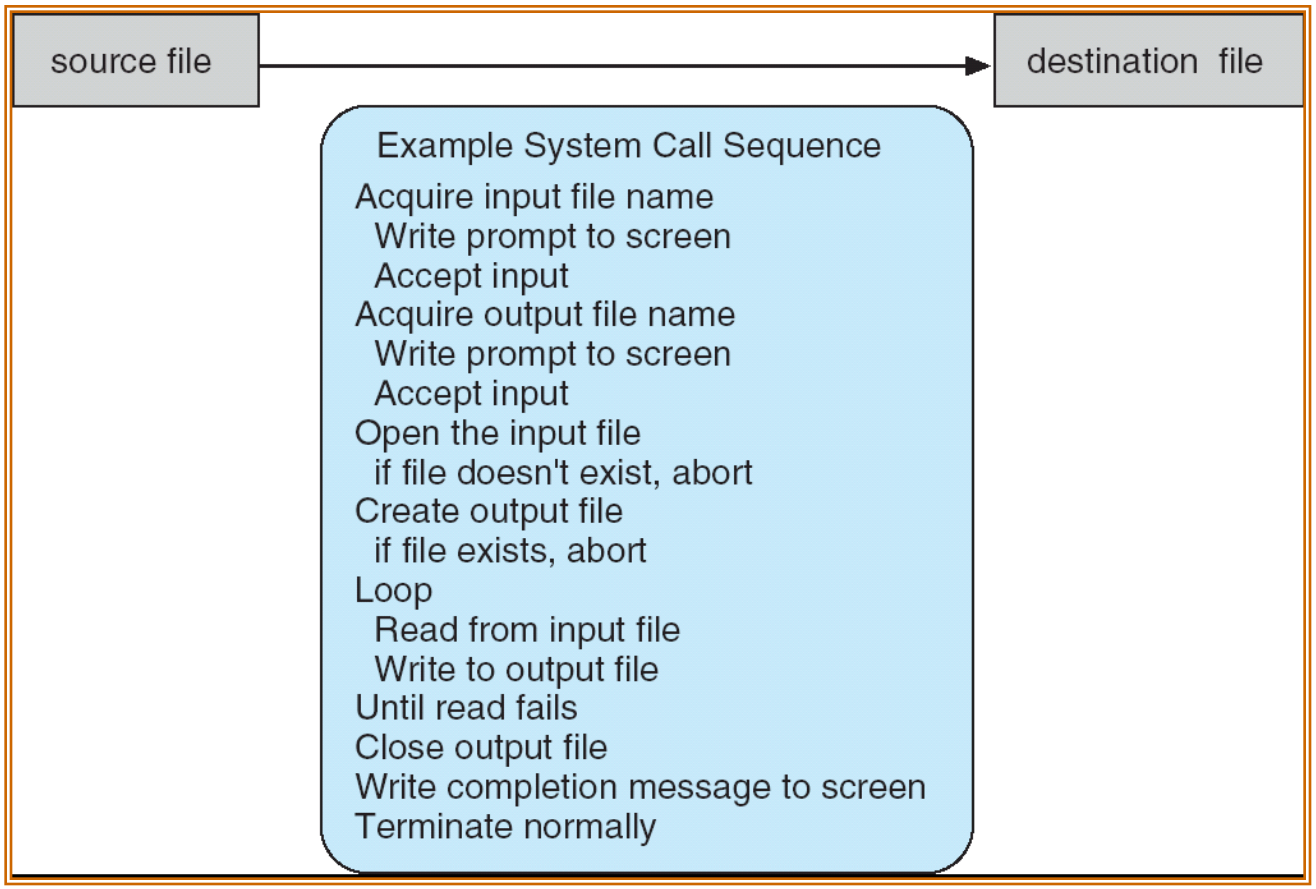
System Calls

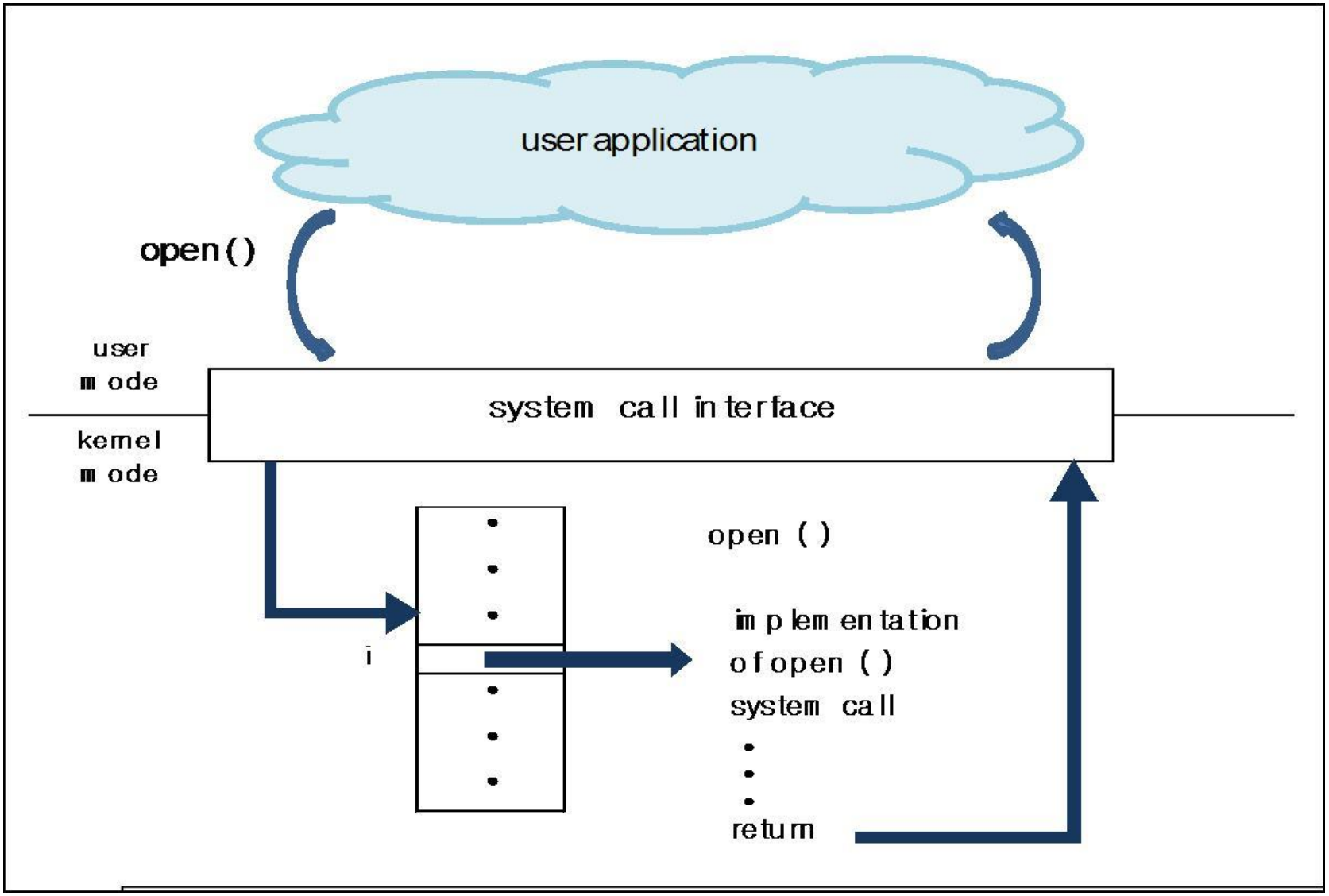
- Provides Interface between Running Program and the Operating System.
- It provides as an interface to the services made available by an OS
  - These calls are generally available as routines written in C and C++, although certain low-level tasks, may need to be written using assembly-language instructions.
- Methods used to Pass Parameters between a Running Program and the Operating System.
  1. Pass Parameters in *Registers*.
  2. Store the Parameters in a Table in Memory, and the Table Address is passed as a Parameter in a Register.
  3. *Push* (store) the Parameters onto the *Stack* by the Program, and *Pop* off the Stack by Operating System.

Example of System Calls

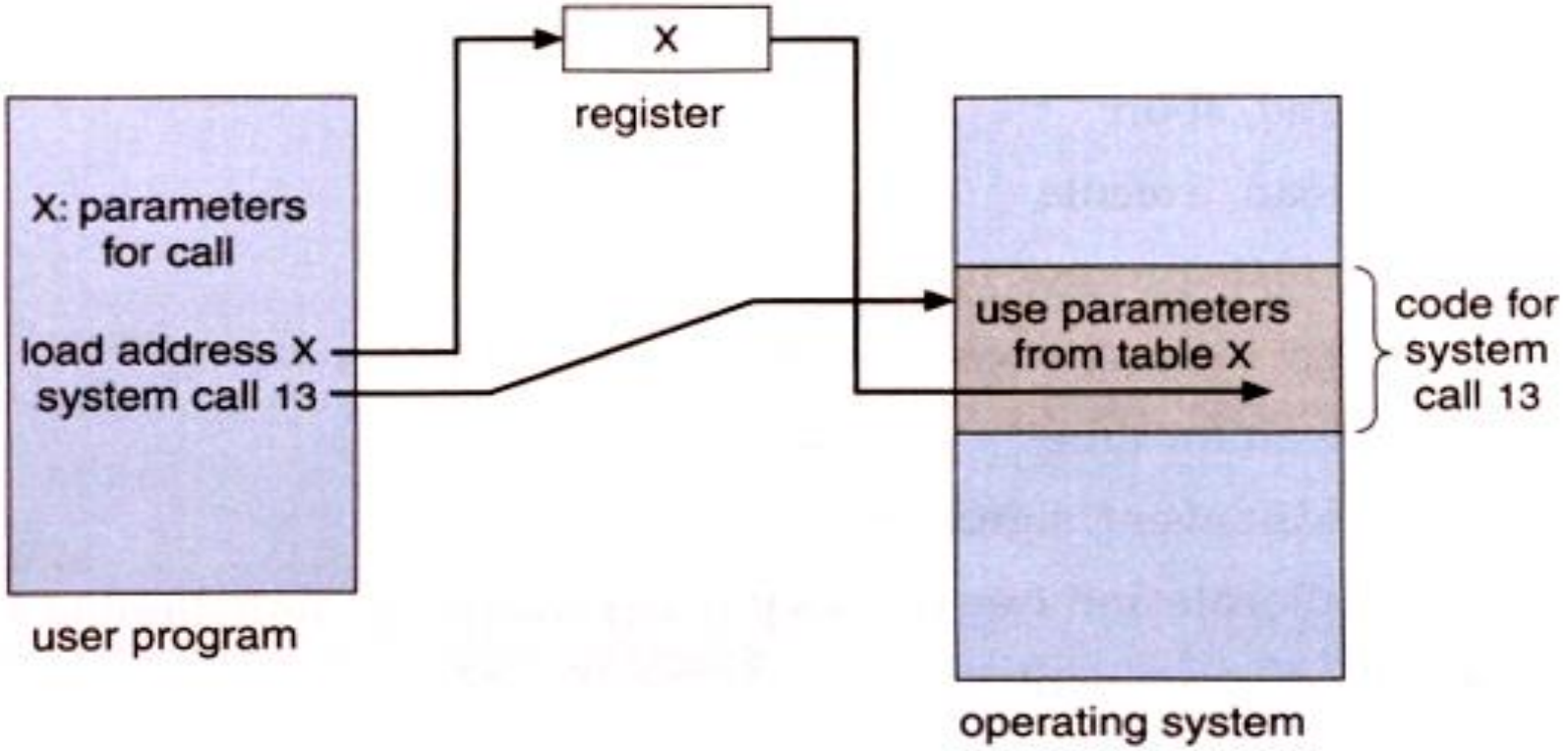
System Call Sequence

to Copy the contents of One file to another file.





The handling of a user application invoking the open() system call



Passing of Parameters as a table

Types of System Calls

|<<

1. **Process Control**
2. **File Management**
3. **Device Management**
4. **Information Maintenance**
5. **Communications**

Example

System Calls – Process Control

- **End, Abort**
- **Load, Execute**
- **Create Process, Terminate Process**
- **Get Process Attributes, Set Process Attributes**
- **Wait for time**
- **Wait Event, Signal Event**
- **Allocate and Free Memory**

System Calls – File Management

|<<

- **Create File, Delete File**
- **Open, Close**
- **Read, Write, Reposition**
- **Get File attributes, Set File attributes**

System Calls – Device Management

- **Request Device, Release Device**
- **Read, Write, Reposition**
- **Get Device attributes, Set Device attributes**
- **Logically Attach or Detach Devices**

System Calls – Information Maintenance

|<<

- **Get Time or Date, Set Time or Date**
- **Get System Data, Set System Data**
- **Get Process, File, or Device attributes**
- **Set Process, File, or Device attributes**

System Calls – Communications

Message Passing Vs Shared Memory

- **Create, Delete Communication connection**
- **Send, Receive Messages**
- **Transfer Status information**
- **Attach or Detach Remote Devices**

System Programs

|<<

System programs provide a convenient environment for program development and execution. They can be divided into:

Categories

1. **Programming Language Support**
2. **Program Loading and Execution**
3. **File Management**
4. **File Modification**
5. **Status information**
6. **Communications**

System Calls

1. **Process Control**
2. **File Management**
3. **Device Management**
4. **Information Maintenance**
5. **Communications**

System Programs – Programming Language Support

- **Compilers**
- **Assemblers**
- **Interpreters**

System Programs – Program Loading & Execution

- **Absolute Loaders**
- **Relocatable Loaders**
- **Linkage Editors**
- **Overlay Loaders**
- **Debugging Systems**

System Programs – File Management

- Create
- Delete
- Copy
- Rename
- Print
- Dump
- List
- Manipulate Files and Directories

System Programs – File modification

Text Editors

To Create and Modify the Contents of Files Stored on Disk or Tape.

Screen Editor – vi

Line Editor – edlin, ed, …

System Programs – Status information

Date

Time

Amount of available Memory / Disk Space

No. of users..

•Some ask the system for info - date, time, amount of available memory, disk space, number of users

•Others provide detailed performance, logging, and debugging information
•Typically, these programs format and print the output to the terminal or other output devices

•Some systems implement a **registry** - used to store and retrieve configuration information

System Programs – Communications

|<<

Provide mechanism for creating virtual connections
among processes, users and different computer systems.

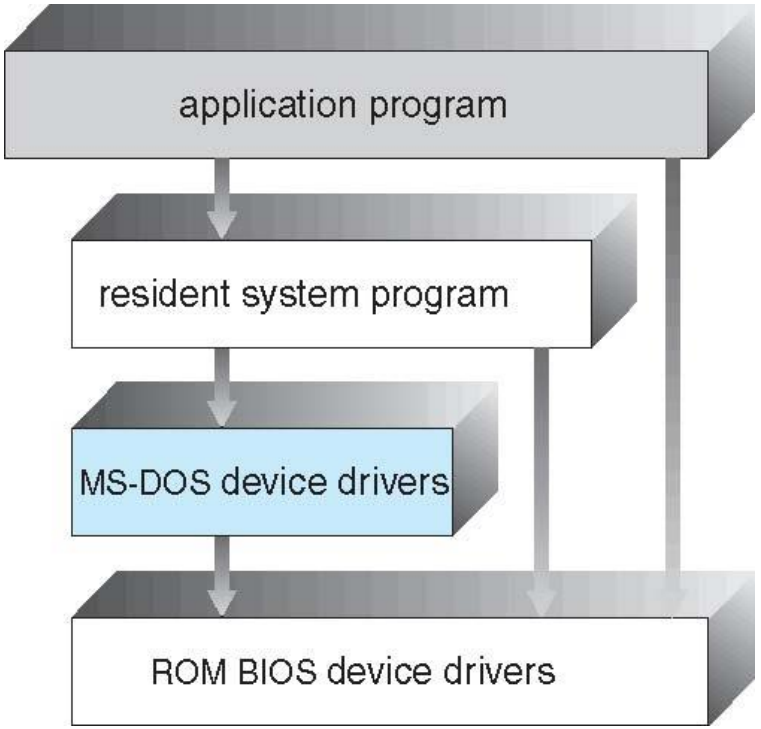
Allow users to
- **Send messages to one another's Screens.**
- **Browse Web pages.**
- **Send Electronic Mail messages.**
- **Log-in Remotely.**
- **Transfer Files from one machine to another.**

Operating System Structure

- General-purpose OS is very large program
- Various ways to structure ones
  - Simple structure – MS-DOS
  - More complex -- UNIX
  - Layered – an abstraction
  - Microkernel -Mach

Simple Structure -- MS-DOS

- MS-DOS – written to provide the most functionality in the least space
  - So, not divided into modules carefully
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



- For instance, application programs are able to access the basic I/O routines to write directly to the display and disk drivers. Such freedom leaves MS-DOS vulnerable to malicious programs, causing entire system crashes when user programs fail.

- Ms-DOS is also limited by the hardware

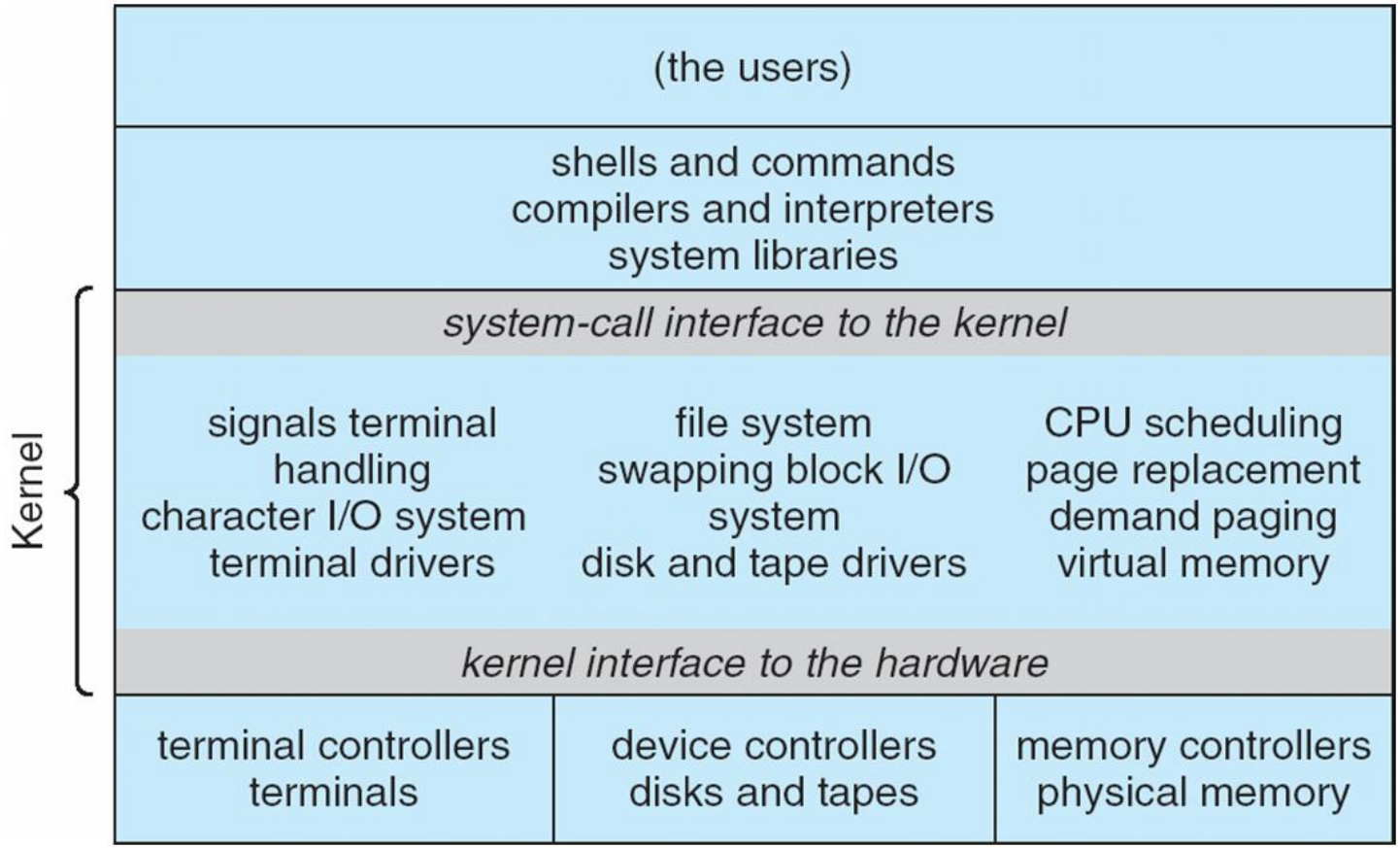
Non Simple Structure -- UNIX

UNIX – initially, limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts

- Systems programs
- The kernel
  - Consists of everything below the system-call interface and above the physical hardware
  - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

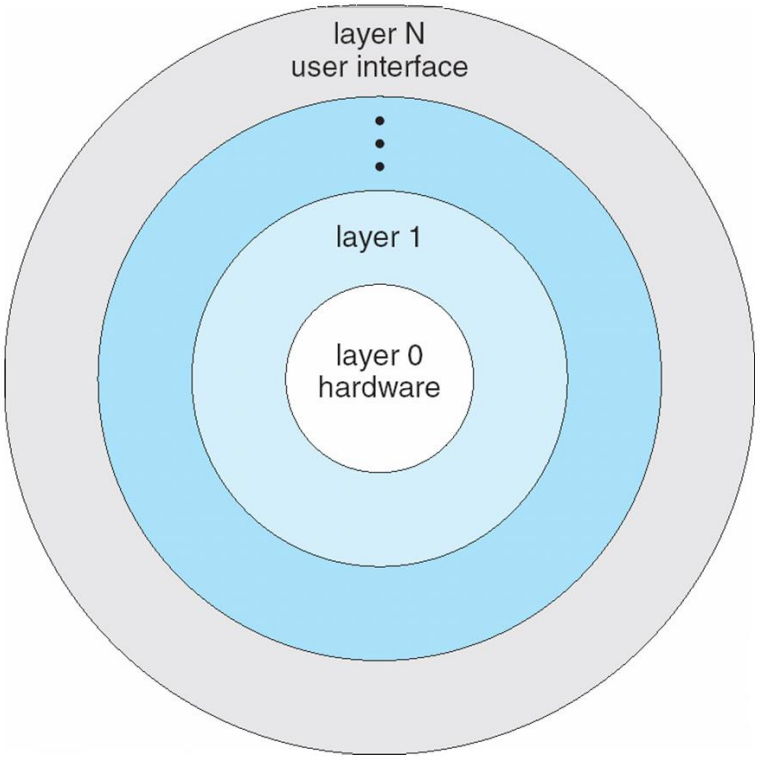
Traditional UNIX System Structure

Beyond simple but not fully layered



Layered Approach

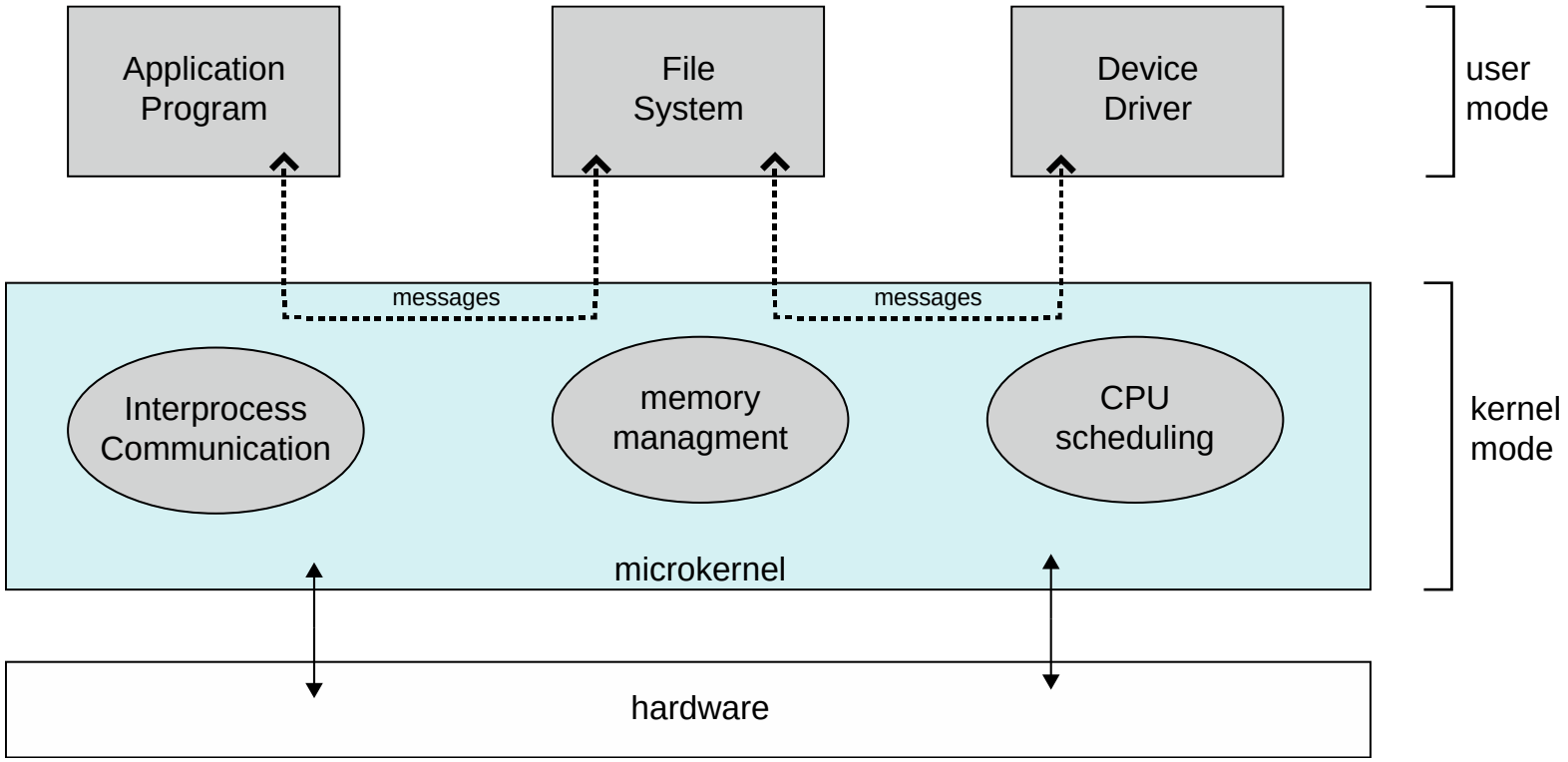
- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers



Microkernel System Structure

- Moves as much from the kernel into user space. This method structures the OS by removing all nonessential components from the kernel and implementing them as system and user-level programs.
- Provide minimal process and memory management.
- **Mach** example of **microkernel**
  - Mac OS X kernel (**Darwin**) partly based on Mach
- Communication takes place between user modules using **message passing**
- Benefits:
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- Detriments:
  - Performance overhead of user space to kernel space communication

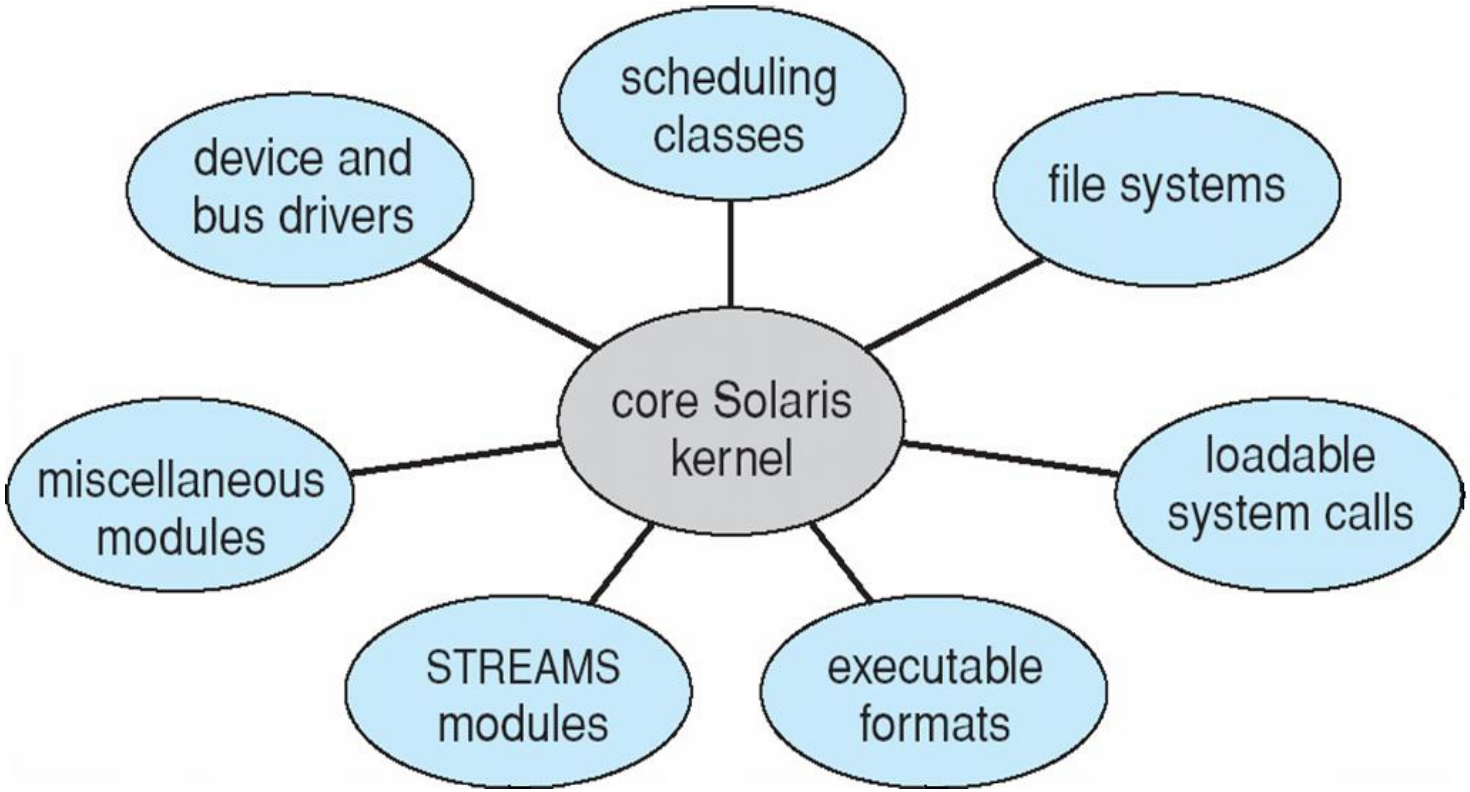
Microkernel System Structure



Modules

- The best current methodology for operating-system design involves object-oriented programming techniques to create a modular kernel.
- Here the kernel has a set of core components and links in additional services during boot time or during run time.
- Many modern operating systems implement **loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc

Solaris Modular Approach



Hybrid Systems

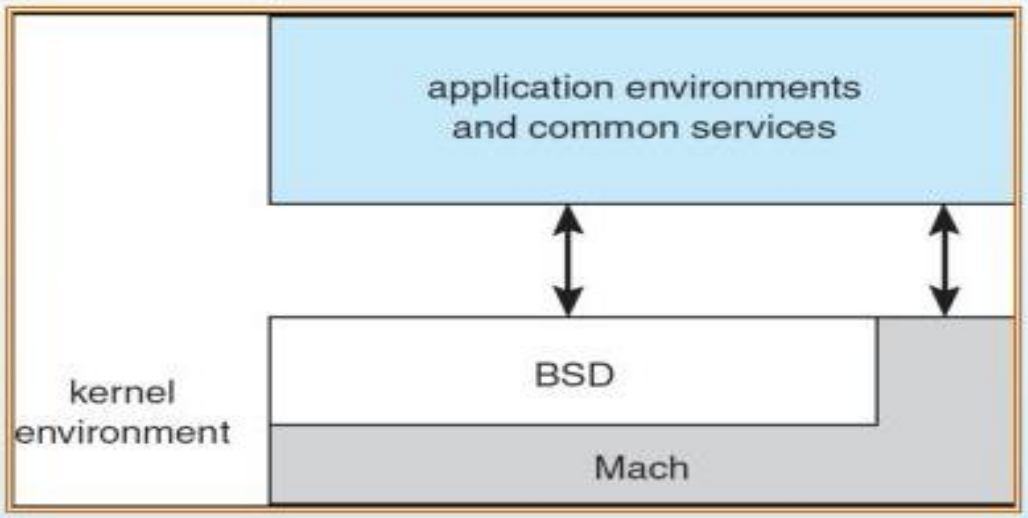
- Most modern operating systems are actually not one pure model
  - Hybrid combines multiple approaches to address performance, security, usability needs
  - Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionality
  - Windows mostly monolithic
- Apple Mac OS X uses hybrid structure. It is a layered system in which one layer consists of the Mach microkernel.
- The top layers include application environments and a set of services providing GI to applications. Below is the kernel environment which consists Mach(memory management,

  RPC, IPC,Thread scheduling, message passing).

  BSD : provides CL interface, networking and file systems

Mac OS X Structure

Apple Mac OS X Structure (hybrid)



Threads

- A thread is a single sequence stream within a process. Threads are also called **lightweight processes** as they possess some of the properties of processes. Each thread belongs to exactly one process.
- In an operating system that supports multithreading, the process can consist of many threads. But threads can be effective only if the CPU is more than 1 otherwise two threads have to context switch for that single CPU.
- All threads belonging to the same process share – code section, data section, and OS resources (e.g. open files and signals)
- But each thread has its own (thread control block) – thread ID, program counter, register set, and a stack
- Any operating system process can execute a thread. we can say that single process can have multiple threads.

Why Do We Need Thread?

- Threads run in parallel improving the application performance. Each such thread has its own CPU state and stack, but they share the address space of the process and the environment.
- Threads can share common data so they do not need to use inter-process communication. Like the processes, threads also have states like ready, executing, etc.
- Priority can be assigned to the threads just like the process, and the highest priority thread is scheduled first.
- Each thread has its own Thread Control Block (TCB). Like the process, a context switch occurs for the thread, and register contents are saved in (TCB). As threads share the same address space and resources, synchronization is also required for the various activities of the thread.

Components of Threads

These are the basic components of the Operating System.

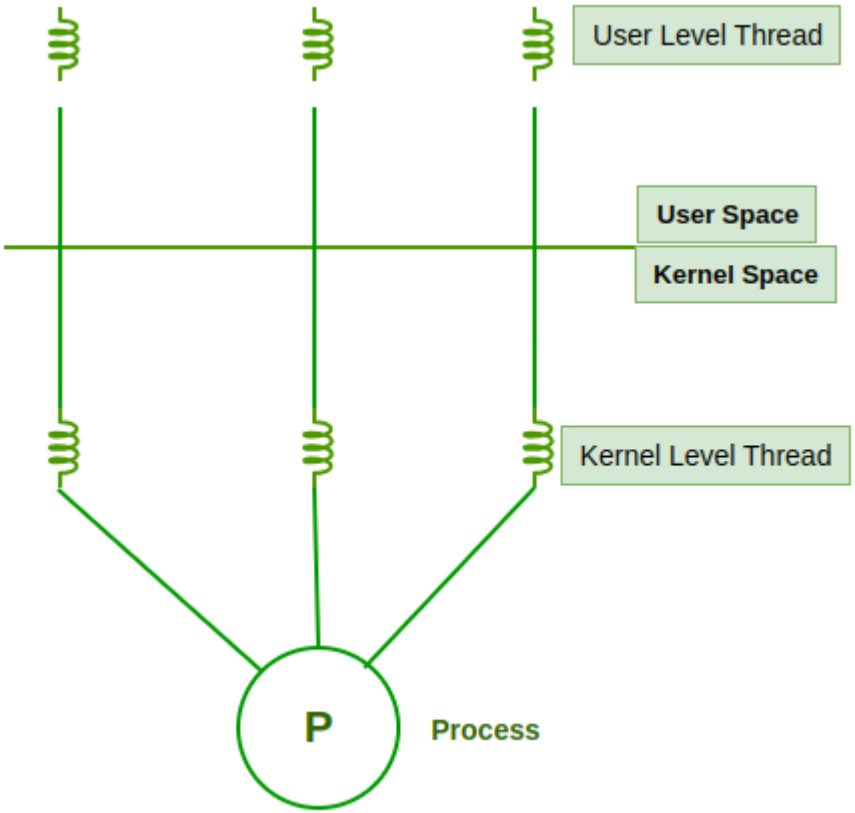
- **Stack Space:** Stores local variables, function calls, and return addresses specific to the thread.
- **Register Set:** Hold temporary data and intermediate results for the thread's execution.
- **Program Counter:** Tracks the current instruction being executed by the thread.

Types of Thread in Operating System

Threads are of two types.

- User Level Thread
- Kernel Level Thread

Fig. User-Level Threads and Kernel-Level Threads



User Level Threads

- The operating system does not recognize the user-level thread. User threads can be easily implemented and it is implemented by the user. If a user performs a user-level thread blocking operation, the whole process is blocked. The kernel level thread does not know nothing about the user level thread. The kernel-level thread manages user-level threads as if they are single-threaded processes.
- Examples: Java thread, POSIX threads, etc.

User Level Threads

Advantages:

The user threads can be easily implemented than the kernel thread.

- It is faster and efficient.
- Context switch time is shorter than the kernel-level threads.
- It does not require modifications of the operating system.
- User-level threads representation is very simple. The register, PC, stack, and mini thread control blocks are stored in the address space of the user-level process.
- It is simple to create, switch, and synchronize threads without the intervention of the process.

Disadvantages of User-level threads

- User-level threads lack coordination between the thread and the kernel.
- If a thread causes a page fault, the entire process is blocked.

Kernel Level Thread

- The kernel-level thread is implemented by the operating system. The kernel knows about all the threads and manages them. The kernel-level thread offers a system call to create and manage the threads from user-space. The implementation of kernel threads is more difficult than the user thread. Context switch time is longer in the kernel thread. If one kernel thread performs a blocking operation, the other kernel thread execution can continue. Example: Window Solaris.

Advantages of Kernel-level threads

- The kernel-level thread is fully aware of all threads..

Disadvantages of Kernel-level threads

- The implementation of kernel threads is difficult than the user thread.
- The kernel-level thread is slower than user-level threads.

Exam Questions

1. **Define *OS*.**
2. **Write about *Evolution of Operating Systems*.**
3. **Explain *basic Structure of a Computer System* & also Explain its *basic elements*.**
4. **Discuss the basic components of a *virtual computer*.**
5. **What are *OS Objectives*?**
6. **What is an *Operating System*? Explain the *functions of OS*.**
7. **Discuss the various *approaches of designing an operating system*.**
8. **Write about *Multitasking*.**
9. **Explain the concept of *Multiprogramming*.**

Exam Questions

|<<

10. **What are the *multi-tasking, multi-programming and multi-threading*?**
11. **Explain the following:**
    a. Multiprogramming
    b. Timesharing
    c. Virtual Memory
12. **What are *Distributed Operating Systems*?**
13. **What are the *types of real-time systems*?**
14. **What is *dispatcher*?**
15. **What is a *system call*? Explain the *categories of system calls*.**
16. **What is a *system program*? Explain different *types of system programs*.**